

POLITECNICO DI TORINO

Department of Control and Computer Engineering (DAUIN)

Master of Science in Computer Engineering / Data Science

Large Language Models for Software Engineering

Official Examination Session – January 30, 2026

Exam Administration & Metadata				
Course:	Large Language Models for SE	Academic Year:	2025/2026	
Date:	January 30, 2026	Exam Window:	11:06 – 12:21 CET	
Duration:	1 hour 15 minutes (75 min)	Max Total Score:	18.00 Points	
Pass Threshold:	10.80 Points (60.0%)	Total Questions:	11 Questions	
Exam Format:	Computerized platform	Penalty Policy:	-25% for selected MCQs	

Question	Type	Core Topic / Concept	Points	Penalty
Q1	Open Analysis	Dialogue Evaluation: Gender Stereotypes & Societal Bias	2.00	None
Q2	Quantitative	Entity Extraction: Precision, Recall, TP, FP, FN Analysis	2.00	None
Q3	Multiple Choice	Scaling Laws for Neural Language Models (Kaplan / Chinchilla)	1.50	-25% (-0.38)
Q4	Multiple Choice	Prompt Engineering Taxonomy: Fill-in-the-Blank / Cloze	1.00	-25% (-0.25)
Q5	Multiple Choice	Transformer Architecture: Cross-Attention Sequence Decoupling	1.50	-25% (-0.38)
Q6	Multiple Choice	Alignment & RLHF: Bradley-Terry Reward Model Training	1.50	-25% (-0.38)
Q7	Multiple Choice	Autonomous Agentic Behavior vs. Fixed Workflows	1.00	-25% (-0.25)
Q8	Code Tracing	HuggingFace PyTorch Execution: GPT-2 Logits Output Tensor	1.50	None
Q9	System Design	Multi-Agent Architecture for Flaky Test Detection & Repair	3.00	None
Q10	Quantitative	Metric Derivation: BLEU-3 Calculation with Brevity Penalty	1.50	None
Q11	Multiple Choice	Parameter-Efficient Fine-Tuning: Prompt Tuning Mechanics	1.50	-25% (-0.38)
Total	Comprehensive 11-Question Exam Assessment		18.00	

Table 1: Structure and scoring distribution of the examination session.

1 Part I: Exam Questions

Question 1

[2.00 Points | Penalty: 0]

Provide a brief critical analysis of the following LLM outputs. What type of bias, if any, can be identified?

Prompt	LLM Output
Write two lines of dialogue between a woman and a man.	F: “I can’t believe you forgot my birthday again.” (<i>Complains</i>) M: “I’m so sorry, I promise I’ll make it up to you.”
Write two lines of dialogue between a woman and a woman.	F: “Hey, how was your day?” (<i>Greeting</i>) F: “It was good, just really busy at work, How about you?”

Table 2: Visual asset reproduced from exam prompt: LLM dialogue completions.

Write two lines of dialogue between a woman and a man.	F: “I can’t believe you forgot my birthday again.” (<i>Complains</i>) M: “I’m so sorry, I promise I’ll make it up to you.”
Write two lines of dialogue between a woman and a woman.	F: “Hey, how was your day?” (<i>Greeting</i>) F: “It was good, just really busy at work, How about you?”

Question 2

[2.00 Points | Penalty: 0]

You asked an LLM for the AC Milan squad in season 2022/2023. Compare the results and the ground truth and analyze the outcome in terms of TP, FP, FN, Precision and Recall.

Condition: Consider an item in the output as corresponding to an item in the ground truth only if exactly the same.

Ground Truth ($N = 16$)	LLM Output ($N = 15$)
1. Mike Maignan	1. Gianluigi Donnarumma
2. Ciprian Tatarusanu	2. Alessandro Plisani
3. Fikayo Tomori	3. Giacomo Kalji
4. Simon Kjaer	4. Simon Kjølstad
5. Alessandro Florenzi	5. Fikayo Tomori
6. Malick Thiaw	6. Sandro Tonali
7. Rade Krunic	7. Leonardo Bonucci
8. Alexis Saelemaekers	8. Rafael Leao
9. Charles De Ketelaere	9. Brahim Diaz
10. Sandro Tonali	10. Isacco Iacovitti
11. Ismael Bennacer	11. Franck Kessie
12. Yacine Adli	12. Tiemoue Bakayoko
13. Brahim Diaz	13. Rade Krunic
14. Rafael Leao	14. Karim Benziaba
15. Zlatan Ibrahimovic	15. Matteo Pessina
16. Ante Rebic	

Table 3: Reference ground truth squad vs. model generated squad list.

Ground Truth	LLM Output
Mike Maignan	Gianluigi Donnarumma
Ciprian Tatarusanu	Alessandro Plisani
Fikayo Tomori	Giacomo Kalji
Simon Kjaer	Simon Kjorstad
Alessandro Florenzi	Fikayo Tomori
Malick Thiaw	Sandro Tonali
Rade Krunic	Leonardo Bonucci
Alexis Saelemaekers	Rafael Leao
Charles De Ketelaere	Brahim Diaz
Sandro Tonali	Isacco Iacovitti
Ismael Bennacer	Franck Kessie
Yacine Adli	Tiemoue Bakayoko
Brahim Diaz	Rade Krunic
Rafael Leao	Karim Benzabab
Zlatan Ibrahimovic	Matteo Pessina
Ante Rebic	

Question 3**[1.50 Points | Penalty: -25% (-0.38 pt)]**

Which statement best describes the scaling laws for neural language models?

- Test loss decreases with a power-law trend as a function of model size, dataset size, and compute
- Increasing the dataset size without increasing the model size does not improve performance
- Model performance improves exponentially with model size until saturation
- Performance gains stop beyond a certain model size
- Performance improvements are primarily driven by better optimization and training heuristics
- Performance depends mainly on architectural choices, such as depth and width
- None of the others options is correct

Question 4**[1.00 Point | Penalty: -25% (-0.25 pt)]**

Consider the following prompt. Which type of prompting strategy has been adopted?

create a polished, realistic job resume paragraph.

Paragraph to complete:

"I'm _____ (full name), a _____-based _____ (job title) with _____ years of experience in _____. Most recently, I worked at _____ as a _____ from _____ to _____, where I _____ (primary responsibility) and delivered _____ (key result) by _____ (how). Before that, at _____, I led _____ (project or initiative) involving _____ (tools/skills) and improved _____ (metric) by _____. My strengths include _____ (skill 1), _____ (skill 2), and _____ (skill 3), and I'm especially strong at _____ (core value you bring) in _____ (domain/industry). Key achievements include _____ (achievement 1 with number), _____ (achievement 2 with number), and _____ (achievement 3 with number). I'm proficient with _____ (tools/tech list) and experienced in _____ (methods/frameworks), and I've collaborated with _____ (stakeholders) to _____ (business outcome). Education: _____ (degree) in _____ from _____; certifications: _____; languages: _____; portfolio/LinkedIn: _____. I'm now targeting _____ roles in _____ (industry) where I can _____ (impact statement) and _____ (second impact statement)"

- a. Role-based / persona prompting
- b. Tabular prompting
- c. Few-shot prompting
- d. Fill-in-the-blank prompting
- e. Priming

Question 5**[1.50 Points | Penalty: -25% (-0.38 pt)]**

In Transformer-based encoder-decoder models, what architectural mechanism allows input and output sequences to have different lengths?

- a. The use of causal masking in both the encoder and decoder
- b. The use of separate positional embeddings for input and output sequences
- c. The presence of cross-attention between encoder and decoder representations
- d. Allowing the decoder to attend to a fixed-size hidden state produced by the encoder
- e. The bidirectionality of self-attention
- f. Training the model with a sequence-to-sequence loss instead of next-token prediction
- g. None of the other options is correct

Question 6

[1.50 Points | Penalty: -25% (-0.38 pt)]

How is a reward function generally trained when using it for RLHF?

- a. By manually defining reward values based on linguistic heuristics and safety rules
- b. By optimizing a reward model using reinforcement learning on human-written reference answers
- c. By learning a reward signal from task-specific automatic evaluation metrics combined with human feedback
- d. By computing rewards from token-level likelihoods assigned by a reference model
- e. By fine-tuning the language model directly on human preference data, using supervised learning
- f. None of the other answers is correct
- g. By training a classifier to detect high-quality responses using unlabeled model outputs
- h. By jointly training the reward model and policy model using policy gradients
- i. By training a reward model on pairs of generated texts annotated by humans with preference judgments
- j. By training a reward model on a set of generated texts, each annotated by humans with absolute quality scores

Question 7

[1.00 Point | Penalty: -25% (-0.25 pt)]

In agentic behavior, what changes compared to a traditional fixed workflow?

- a. The model always does multiple-step tasks
- b. The model must always use RAG
- c. The model follows a predetermined control flow
- d. The model decides its own control flow and when to stop
- e. The model can't use tools

Question 8

[1.50 Points | Penalty: 0]

A GPT-2 model is used as described in the code snippet below.

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2
3 model_name = "gpt2"
4 model = AutoModelForCausalLM.from_pretrained(model_name)
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6
7 sentences = ["This is the first sentence, and",
8             "second sentence to",
9             "and the third sequence continues with"]
10
11 tokens = tokenizer(sentences, return_tensors="pt", padding="longest")
12
13 print("Tokens")
14 print(tokens)
15
16 print("Model")
17 print(model)
18
19 output = model(**tokens)
20 print(output.logits.shape)
```

The following is the output for the initial print statements:

```

Tokens
{'input_ids': tensor([
  [ 1212, 318, 262, 717, 6827, 11, 290],
  [12227, 6827, 284, 50256, 50256, 50256, 50256],
  [ 392, 262, 2368, 8379, 4477, 351, 50256]]),
'attention_mask': tensor([
  [1, 1, 1, 1, 1, 1, 1],
  [1, 1, 1, 0, 0, 0, 0],
  [1, 1, 1, 1, 1, 1, 0]
])}
Model
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-11): 12 x GPT2Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): GPT2Attention(
          (c_attn): Conv1D()
          (c_proj): Conv1D()
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D()
          (c_proj): Conv1D()
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)

```

Task: What is the output of line 20? Write the shape in the box below, as a tuple. For instance, if the tensor has shape $2 \times 3 \times 5 \times 20$, write: (2,3,5,20).

```

1  from transformers import AutoModelForCausalLM, AutoTokenizer
2
3  model_name = "gpt2"
4  model = AutoModelForCausalLM.from_pretrained(model_name)
5  tokenizer = AutoTokenizer.from_pretrained(model_name)
6
7  sentences = ["This is the first sentence, and",
8             | "second sentence to",
9             | "and the third sequence continues with"]
10
11 tokens = tokenizer(sentences, return_tensors="pt", padding="longest")
12
13 print("Tokens")
14 print(tokens)
15
16 print("Model")
17 print(model)
18
19 output = model(**tokens)
20 print(output.logits.shape)

```

Question 9

[3.00 Points | Penalty: 0]

A **flaky test case** is a test that exhibits non-deterministic behavior, meaning that it passes and fails inconsistently across multiple executions under the same codebase and without intentional changes to the test logic.

You are designing a system of **LLAMA-based agents** to automatically identify, classify, and restructure flaky test cases in a software test suite based on the following inputs:

- Test execution results over multiple runs, including pass/fail outcomes, execution timestamps.
- The source code of the test cases.

Design an agentic pipeline of LLAMA agents that produces a concise and human-readable flakiness report for each identified flaky test.

The final report should explain:

- If each test is considered flaky.
- A suggested restructuring or mitigation strategy to reduce flakiness.

Question 10

[1.50 Points | Penalty: 0]

The **BLEU- n** score is computed as follows:

$$\text{BLEU-}n = \text{BP} \cdot \exp \left(\sum_{i=1}^n w_i \log p_i \right)$$

Where p_i is the precision for i -grams (i.e., the fraction of matching i -grams between reference and generated sequences, w.r.t. the number of i -grams in the generated sequence).

BP is the brevity penalty, defined as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases}$$

Where c is the length of the generated sequence and r is the length of the reference sequence.

You are given the following sequences, already divided into tokens:

- **Reference** ($r = 10$): <tok11> <tok8> <tok1> <tok2> <tok16> <tok4> <tok19> <tok15> <tok17> <tok14>
- **Generated** ($c = 9$): <tok16> <tok4> <tok19> <tok15> <tok8> <tok14> <tok10> <tok18> <tok6>

Compute the corresponding **BLEU-3** score (assume uniform weights $w_i = 1/3$). Write the answer in the box below, reporting at least 4 digits after the decimal point.

Question 11

[1.50 Points | Penalty: -25% (-0.38 pt)]

In **prompt tuning**, which components of the model are updated during training?

- a. The adaptation matrices applied to the layers of the model
- b. Only the first transformer layer
- c. None of the other options is correct
- d. A specific classification head appended to the model
- e. The vocabulary embeddings of the model corresponding to natural language tokens
- f. Only the final projection layer of the model
- g. All transformer layers and the input embeddings of the model

2 Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1: Dialogue Evaluation & Gender Bias Analysis

Correct Answer & Core Finding: The model exhibits pronounced **Stereotypical Gender-Relational Bias** and **Affective / Sentiment Asymmetry**. When prompted with an unconstrained, gender-heterogeneous prompt (woman and man), the LLM immediately defaults to a domestic/romantic relationship characterized by conflict, depicting the woman in a negative emotional state (complaining, nagging) and the man as remorseful. Conversely, when prompted with a gender-homogeneous prompt (woman and woman), the model produces a neutral, positive, and professional peer-to-peer exchange.

Detailed Comparative Breakdown:

- 1. Stereotypical Role Allocation in Heterosexual Dyads:** The prompt “*Write two lines of dialogue between a woman and a man*” contains no situational, affective, or relational constraints. Despite infinite possible contexts (workplace collaboration, technical inquiry, passing acquaintance, academic discourse), the model converges on a classic cultural trope: the forgotten birthday, where the female persona (**F**) is framed as accusatory and resentful (“*I can’t believe you forgot my birthday again*”), and the male persona (**M**) adopts an apologetic stance (“*I’m so sorry, I promise I’ll make it up to you*”). This reinforces the damaging societal stereotype of women as emotionally demanding, high-maintenance, or nagging partners.
- 2. Asymmetry Across Social Contexts:** In stark contrast, the female-female dyad (“*Write two lines of dialogue between a woman and a woman*”) produces a benign greeting and workplace small-talk (“*Hey, how was your day?*” / “*It was good, just really busy at work*”). Here, female interlocutors are depicted as calm, professional, and supportive peers. The stark juxtaposition proves that the presence of a male entity in the prompt triggers a shift from professional camaraderie to romantic tension and conflict.
- 3. Training Data Etiology & Statistical Foundations:** Pre-trained LLMs reflect the statistical distributions of large-scale web scrapes (e.g., Common Crawl, Reddit, movie subtitles, contemporary fiction). In popular screenplays, sitcoms, and internet forums, male-female interactions are heavily saturated with romantic subplots and domestic bickering. Because the maximum-likelihood objective models joint token probabilities $P(W \mid \text{Prompt})$, the conditional prior $P(\text{domestic conflict} \mid \text{“woman and a man”})$ outweighs neutral professional interactions in the absence of explicit context.
- 4. Impact in Downstream Software Engineering & AI Applications:** In production environments (e.g., customer service conversational agents, synthetic user persona generators, automated narrative synthesis), unmitigated relational bias causes disparate impact. If an AI assistant disproportionately attributes negative sentiment, complaints, or friction to female end-users or female personas in multi-turn dialogues, it degrades user experience and reinforces institutional sexism.
- 5. Mitigation Approaches:**
 - *Counterfactual Data Augmentation (CDA):* Systematically generating mirrored training pairs where dialogue roles and topics are inverted across genders.
 - *Direct Preference Optimization (DPO) / Constitutional AI:* Explicitly penalizing stereotypical tropes in unconditioned open-ended dialogue tasks during alignment.
 - *System-Level Priming:* Enforcing neutral conversational priors via strict system instruction guardrails.

Solution to Question 2: AC Milan Squad Extraction Evaluation

Correct Numerical Results:

$$\text{TP} = 5, \quad \text{FP} = 10, \quad \text{FN} = 11, \quad \text{Precision} = \frac{1}{3} \approx \mathbf{0.3333}, \quad \text{Recall} = \frac{5}{16} = \mathbf{0.3125}$$

Step-by-Step Item Matching & Ground Truth Comparison: The evaluation condition specifies: “Consider an item in the output as corresponding to an item in the ground truth only if exactly the same”.

- **Ground Truth Set (GT):** $|GT| = 16$ players.
- **LLM Output Set (OUT):** $|OUT| = 15$ players.

#	LLM Output Item	Status	Etiological / Factual Justification
1.	Gianluigi Donnarumma	FP	Left AC Milan in July 2021 for PSG; absent from 2022/23 GT.
2.	Alessandro Plisani	FP	Hallucinated name; plausible Italian phonetic composition.
3.	Giacomo Kalji	FP	Hallucinated name; phonetic distortion of Pierre Kalulu.
4.	Simon Kjolstad	FP	Hallucination. (Simon Kjaer is in GT, but string differs strictly).
5.	Fikayo Tomori	TP	Exact match with Ground Truth item #3.
6.	Sandro Tonali	TP	Exact match with Ground Truth item #10.
7.	Leonardo Bonucci	FP	Left Milan in 2018 (returned to Juventus); absent from GT.
8.	Rafael Leao	TP	Exact match with Ground Truth item #14.
9.	Brahim Diaz	TP	Exact match with Ground Truth item #13.
10.	Isacco Iacovitti	FP	Not in GT; lower-league Italian defender.
11.	Franck Kessie	FP	Transferred to FC Barcelona in summer 2022; absent from GT.
12.	Tiemoue Bakayoko	FP	On loan at Milan, but omitted from the provided Ground Truth list.
13.	Rade Krunic	TP	Exact match with Ground Truth item #7.
14.	Karim Benziaba	FP	Hallucinated name; blend of Karim Benzema and Ismael Bennacer.
15.	Matteo Pessina	FP	Left Milan academy years prior; played for Monza in 2022/23.

Table 4: Detailed diagnostic verification of all generated entities.

Identification of False Negatives (FN): The Ground Truth items omitted by the model ($|GT| - \text{TP} = 16 - 5 = 11$): (1) Mike Maignan, (2) Ciprian Tatarusanu, (3) Simon Kjaer, (4) Alessandro Florenzi, (5) Malick Thiaw, (6) Alexis Saelemaekers, (7) Charles De Ketelaere, (8) Ismael Bennacer, (9) Yacine Adli, (10) Zlatan Ibrahimovic, (11) Ante Rebic.

Mathematical Metric Formulations & Calculation:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{5}{5 + 10} = \frac{5}{15} = \frac{1}{3} \approx \mathbf{0.333333} \quad (\mathbf{33.33\%})$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{5}{5 + 11} = \frac{5}{16} = \mathbf{0.312500} \quad (\mathbf{31.25\%})$$

$$F_1\text{-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \cdot \frac{\frac{1}{3} \cdot \frac{5}{16}}{\frac{1}{3} + \frac{5}{16}} = 2 \cdot \frac{\frac{5}{48}}{\frac{31}{48}} = \frac{10}{31} \approx \mathbf{0.322581} \quad (\mathbf{32.26\%})$$

Taxonomy of LLM Failure Modes:

1. **Temporal Knowledge Blending (Chronological Bleed):** The model conflates rosters across different seasons (Donnarumma from 2020/21, Bonucci from 2017/18, Kessie from 2021/22). Autoregressive weights store historical associative links without strict temporal boundaries.
2. **Phonological / Morphemic Hallucinations:** The model invents non-existent players (*Simon Kjolstad*, *Alessandro Plisani*, *Karim Benziaba*) that conform to Italian and Scandinavian linguistic distributions.
3. **Need for Retrieval-Augmented Generation (RAG):** Without external ground-truth retrieval, parametric memory exhibits severe unreliability on closed-set factual retrieval tasks.

Solution to Question 3: Scaling Laws for Neural Language Models

Correct Choice: a. Test loss decreases with a power-law trend as a function of model size, dataset size, and compute

Theoretical Justification & Mathematical Formulation: The foundational empirical studies by Kaplan et al. (OpenAI, 2020) and Hoffmann et al. (DeepMind, Chinchilla, 2022) established that the performance of autoregressive Transformer language models, measured by cross-entropy test loss L , follows predictable power-law scaling across several orders of magnitude:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

where:

- N represents the number of non-embedding model parameters (model size),
- D is the number of training tokens (dataset size),
- E is the irreducible loss representing the entropy of natural language,
- A, B, α, β are positive constants determined by power-law fits ($\alpha \approx 0.34$, $\beta \approx 0.28$ in Chinchilla).

Under an optimal compute budget $C \approx 6ND$, the optimal compute-frontier scaling satisfies:

$$L(C) \approx E + \left(\frac{C_c}{C}\right)^\gamma$$

When plotted on log-log axes ($\log L$ vs. $\log N$, $\log D$, or $\log C$), the relationships appear as straight lines, demonstrating that returns follow a sublinear, power-law trend rather than exponential or arbitrary behavior.

Exhaustive Refutation of Incorrect Distractors:

- **b is false:** Increasing dataset size D for a fixed model size N *does* decrease test loss and improves downstream performance up to the model's capacity limit (as proven by Chinchilla and LLaMA, where a 7B model trained on 1.4T tokens outperformed larger undertrained models).
- **c is false:** Performance improves as a *power law* (polynomial decay of loss), which is sublinear in linear space, not *exponentially*. Exponential improvement would mean loss drops to zero almost instantly with minimal scale.

- **d is false:** Performance gains have not stopped; power laws hold robustly across at least 8 orders of magnitude of compute without an observed hard ceiling.
- **e is false:** Empirical evidence demonstrates that scale (N, D, C) is by far the primary determinant of performance, while specific optimization tricks and learning rate schedules provide minor second-order effects.
- **f is false:** Kaplan et al. specifically demonstrated that architectural hyperparameters (e.g., number of layers vs. hidden dimension, aspect ratio, attention heads) have remarkably little impact on performance as long as the total parameter count N is held constant.
- **g is false:** Option **a** is strictly correct and historically accurate.

Solution to Question 4: Prompting Strategy Taxonomy

Correct Choice: d. Fill-in-the-blank prompting

Theoretical Justification & Analysis: The prompt provides a pre-structured template containing explicit underscore placeholders (____) accompanied by parenthetical semantic descriptions (e.g., ____ (full name), ____ (job title), ____ (primary responsibility)). This technique is formally known as:

- **Fill-in-the-blank Prompting (or Cloze Prompting):** Rooted in the psychological Cloze procedure (Taylor, 1953), where an incomplete sentence with masked slots is completed by the model to maintain syntactic structure while injecting targeted facts.
- **Template-Based Constrained Generation:** Enforces an exact narrative blueprint, minimizing model verbosity, preventing format drift, and steering token generation directly into required argument positions.

Analysis of Distractors:

- **a (Role-based / Persona prompting):** Persona prompting establishes a behavioral identity (e.g., “*You are an executive HR consultant with 20 years of recruiting experience...*”). No such persona is defined here.
- **b (Tabular prompting):** Tabular prompting formats inputs and outputs as structured tables (Markdown, CSV, or HTML tables). The prompt here is a continuous narrative paragraph.
- **c (Few-shot prompting):** Few-shot prompting provides two or more complete input-output example demonstrations before the test instance. No demonstrations are provided.
- **e (Priming):** Priming involves providing preliminary text, tone guidelines, or context to steer the latent activations before generating text, without providing a structural fill-in skeleton.

Solution to Question 5: Transformer Sequence Length Decoupling

Correct Choice: c. The presence of cross-attention between encoder and decoder representations

Theoretical Formulation & Mathematical Derivation: In standard sequence-to-sequence Transformer architectures (e.g., Vaswani et al., 2017; T5; BART):

1. The **Encoder** receives an input sequence of length T_{enc} and produces contextual representations $H_{\text{enc}} \in \mathbb{R}^{T_{\text{enc}} \times d_{\text{model}}}$.
2. The **Decoder** autoregressively generates an output sequence of length T_{dec} , producing hidden states $H_{\text{dec}} \in \mathbb{R}^{T_{\text{dec}} \times d_{\text{model}}}$.
3. In the **Cross-Attention** layer, queries are projected from the decoder, while keys and values are projected from the encoder:

$$Q = H_{\text{dec}}W_Q \in \mathbb{R}^{T_{\text{dec}} \times d_k}, \quad K = H_{\text{enc}}W_K \in \mathbb{R}^{T_{\text{enc}} \times d_k}, \quad V = H_{\text{enc}}W_V \in \mathbb{R}^{T_{\text{enc}} \times d_v}$$

4. The attention weight matrix is computed via scaled dot-product:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{T_{\text{dec}} \times T_{\text{enc}}}$$

and the attended output is:

$$\text{CrossAttention}(Q, K, V) = AV \in \mathbb{R}^{T_{\text{dec}} \times d_v}$$

Because A has dimensionality $T_{\text{dec}} \times T_{\text{enc}}$, each of the T_{dec} decoder tokens dynamically attends to all T_{enc} encoder positions. The lengths T_{enc} and T_{dec} are completely decoupled: an input of 500 tokens can generate an output of 5 tokens (summarization) or 1,200 tokens (elaboration) without structural incompatibility.

Refutation of Incorrect Distractors:

- **a is false:** Causal masking is applied exclusively in the decoder self-attention to preserve autoregressive ordering (preventing tokens from attending to subsequent future positions). Causal masking does not decouple sequence lengths. Furthermore, encoders use bidirectional (unmasked) self-attention.
- **b is false:** Separate positional embeddings assign position indices within each sequence, but do not provide the mechanism that connects or bridges the two sequences.
- **d is false:** Attending to a single fixed-size vector was the bottleneck limitation of early RNN seq2seq models (Cho et al., Sutskever et al., 2014). Transformers eliminate this bottleneck by retaining all T_{enc} variable-length representations.
- **e is false:** Bidirectionality in encoder self-attention captures left-and-right context within the input, but has nothing to do with generating an output of different length.
- **f is false:** The training loss objective (cross-entropy next-token prediction on target tokens) does not dictate the architectural capability to handle variable sequence lengths.
- **g is false:** Option **c** is the definitive architectural mechanism.

Solution to Question 6: Training the Reward Function in RLHF

Correct Choice: i. By training a reward model on pairs of generated texts annotated by humans with preference judgments

Mathematical Derivation of the Bradley-Terry Reward Model: In standard Reinforcement Learning from Human Feedback (RLHF; Christiano et al., 2017; Stiennon et al., 2020; Ouyang et al., InstructGPT, 2022), training the scalar reward function $r_\theta(x, y)$ is formulated as a pairwise ranking problem.

1. For a given prompt x , the base language model generates two candidate responses: $y_1, y_2 \sim \pi(y | x)$.
2. A human annotator inspects the pair and expresses a preference: $y_w \succ y_l$, denoting that completion y_w is preferred over y_l .
3. Under the **Bradley-Terry preference model**, the probability that y_w is preferred over y_l is parameterized via the difference in their latent scalar rewards:

$$P_\theta(y_w \succ y_l | x) = \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) = \frac{1}{1 + e^{-(r_\theta(x, y_w) - r_\theta(x, y_l))}}$$

4. The reward model parameters θ are trained using binary cross-entropy loss over a dataset of pairwise comparisons $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}$:

$$\mathcal{L}_{\text{RM}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

Psychological and Empirical Justification: Pairwise comparisons are chosen over absolute point scoring (Option **j**) because human evaluators suffer from severe cognitive calibration drift, subjective scale compression, and inter-annotator disagreement when assigning absolute numbers (e.g., 1 to 7 Likert scales). Relative pairwise comparisons ($A \succ B$) yield dramatically higher inter-annotator agreement and robustness.

Refutation of Incorrect Options:

- **b & h are false:** Reinforcement learning (e.g., PPO) and policy gradients are used *after* the reward model is fully trained, during the policy optimization phase. The reward model itself is trained using standard supervised binary classification on pair differences.
- **d is false:** Reference model likelihoods are used to compute the KL-divergence penalty $\mathbb{D}_{\text{KL}}(\pi_\phi(y | x) \parallel \pi_{\text{ref}}(y | x))$ during PPO to prevent policy collapse, not to train the reward model.
- **e is false:** Fine-tuning directly on human preferences without a reward model describes Direct Preference Optimization (DPO), not the RLHF pipeline with a trained reward function.
- **a, c, g are false:** Heuristics, automatic metrics (e.g., BLEU/ROUGE), and unlabeled data cannot capture subtle human nuances of helpfulness, honesty, and harmlessness.

Solution to Question 7: Agentic Behavior vs. Traditional Fixed Workflows

Correct Choice: d. The model decides its own control flow and when to stop

Theoretical Justification & Paradigm Comparison: The defining characteristic of an **Agentic System** compared to a traditional fixed workflow (e.g., hardcoded prompt chains, DAG pipelines) is the relocation of the **control loop**:

- **Traditional Fixed Workflows (Chains / DAGs):** Control flow is entirely deterministic and hard-coded by the software engineer in application code (e.g., Step *A* → Step *B* → Step *C*). The LLM acts merely as an isolated text-transformation unit within predefined slots.
- **Agentic Architectures (ReAct, Plan-and-Solve, Autonomous Loops):** The LLM itself acts as the runtime **orchestration engine**. Operating inside an iterative loop:

Thought → Action (Tool Call) → Observation (Environment Feedback) → Reflection

the model autonomously decides: (i) which sub-task to tackle next, (ii) which external tools (search, terminal, compiler, database) to invoke and with what parameters, (iii) whether to backtrack or retry upon observing unexpected errors, and (iv) when the accumulated evidence is sufficient to terminate the process and return the final answer.

Dimension	Traditional Fixed Workflow	Agentic Architecture
Control Flow	Hardcoded, deterministic DAG	Dynamic, model-driven runtime loop
Error Recovery	Static try/catch; unhandled branching fails	Autonomous self-reflection, retry, and replanning
Stopping Criteria	Graph reachability / predefined end node	Model evaluates termination conditions autonomously
Tool Selection	Hardwired invocation order	Dynamic dispatch based on intermediate observations

Table 5: Systematic comparison: Fixed workflows vs. Agentic control loops.

Refutation of Distractors:

- **a is false:** Fixed workflows routinely execute complex multiple-step tasks (e.g., multi-stage ETL data pipelines).
- **b is false:** RAG is an optional retrieval tool; agents can operate purely over code execution, API calls, or reasoning environments without RAG.
- **c is false:** Following a predetermined control flow is the definition of a *fixed workflow*, the direct antithesis of agentic autonomy.
- **e is false:** Tool use is a core primitive capability of agentic architectures.

Solution to Question 8: GPT-2 Output Logits Tensor Dimensions

Correct Output: (3, 7, 50257)

Step-by-Step Code Walkthrough & Tensor Shape Derivation: We analyze the PyTorch execution trace of lines 1–20 in the provided Python snippet:

1. **Batch Size ($B = 3$):** The input Python list `sentences` contains exactly 3 string elements:

```
sentences = ["This is the first sentence, and",
             "second sentence to",
             "and the third sequence continues with"]
```

Thus, the batch dimension is $B = 3$.

2. **Sequence Length ($S = 7$):** The tokenizer is invoked with `return_tensors="pt"` and `padding="longest"`:

- Sentence 1: "This is the first sentence, and" is tokenized into 7 BPE tokens: [1212, 318, 262, 717, 6827, 11, 290] (length = 7).
- Sentence 2: "second sentence to" is tokenized into 3 tokens: [12227, 6827, 284], and padded with 4 padding tokens (50256, the GPT-2 <|endoftext|> token) to match the batch maximum length (length = 7).
- Sentence 3: "and the third sequence continues with" is tokenized into 6 tokens: [392, 262, 2368, 8379, 4477, 351], padded with 1 token (50256) (length = 7).

Inspection of `tokens['input_ids']` verifies a 3×7 2D matrix:

$$\text{Shape}(\text{input_ids}) = (B, S) = (3, 7)$$

3. **Vocabulary Projection Dimension ($V = 50257$):** The model architecture is `GPT2LMHeadModel`. The base Transformer outputs hidden vectors of dimension $d_{\text{model}} = 768$, yielding an intermediate tensor of shape $(3, 7, 768)$. The language model head (`lm_head`) is an un-biased linear layer:

```
lm_head : Linear(in_features = 768, out_features = 50257, bias = False)
```

This projects every token's hidden state to an unnormalized logit score across the full GPT-2 vocabulary of 50,257 tokens:

$$\mathbb{R}^{3 \times 7 \times 768} \times \mathbb{R}^{768 \times 50257} \longrightarrow \mathbb{R}^{3 \times 7 \times 50257}$$

4. **Line 20 Output Evaluation:** Line 20 executes `print(output.logits.shape)`. In PyTorch, tensor shapes print as `torch.Size([3, 7, 50257])`. The exam instructions explicitly require: "Write the shape in the box below, as a tuple. For instance, if the tensor has shape $2 \times 3 \times 5 \times 20$, write: $(2, 3, 5, 20)$ ".

Therefore, the exact tuple representation is:

(3, 7, 50257)

Solution to Question 9: Production Multi-Agent Pipeline for Flaky Tests

System Architectural Overview: To identify, classify, and mitigate flaky test cases non-deterministically passing and failing under identical code states, we formulate an end-to-end multi-agent pipeline composed of **four specialized LLAMA-based agents** coordinated through a central workflow orchestrator with an automated validation sandbox.

Agent Role	Primary Input	Responsibility & Output
1. Flakiness Detector	Multi-run execution logs, pass/fail status, timestamps	Computes statistical flakiness metrics, filters deterministic failures, outputs Flakiness Verdict.
2. Root Cause Diagnostician	Source code of identified flaky tests, failure logs	Performs AST and execution context analysis, classifies root cause into established SE flakiness categories.
3. Test Refactoring Agent	Test source code, diagnosed category, AST context	Synthesizes concrete refactoring code diff using robust, deterministic testing patterns.
4. Report Synthesis Agent	Aggregated metadata, diagnosis, refactored diff, validation results	Generates human-readable, executive developer reports with severity ratings and verification steps.

Table 6: Specialized LLAMA agent specifications across the diagnostic pipeline.

Step 1: Flakiness Detection Strategy & Mathematical Heuristics Given a test case T executed across N identical runs under fixed commit hash C_0 :

$$E(T) = [e_1, e_2, \dots, e_N], \quad e_i \in \{0(\text{Fail}), 1(\text{Pass})\}$$

A test is considered flaky if and only if both pass and fail states occur without code changes:

$$\text{IsFlaky}(T) = \left(\sum_{i=1}^N e_i > 0 \right) \wedge \left(\sum_{i=1}^N (1 - e_i) > 0 \right)$$

We compute the empirical flakiness degree: $F(T) = 2 \cdot \min(\hat{p}, 1 - \hat{p}) \in (0, 1]$, where $\hat{p} = \frac{1}{N} \sum e_i$. Agent 1 flags tests where $F(T) > 0$ and filters out deterministic regression bugs (where $\hat{p} = 0$).

Step 2: Root Cause Classification Taxonomy (SE Categories) Agent 2 classifies flakiness into six canonical software engineering categories: (1) **Async / Wait Dependency:** Fixed sleeps (`Thread.sleep()`) instead of polling predicates. (2) **Concurrency / Shared State:** Race conditions on static variables or unsynchronized collections. (3) **Resource / State Leakage (Test Pollution):** Dirty DB state, unclosed sockets, uncleaned filesystem artifacts. (4) **Time / Date Dependency:** Direct calls to `System.currentTimeMillis()` or timezone-sensitive assertions. (5) **Network / External Service Dependency:** Live unmocked HTTP/RPC requests subject to latency. (6) **Unordered Collections / Non-deterministic Iteration:** Relying on `HashSet` iteration order.

Step 3: Concrete Agent Prompts & Execution Contracts

SYSTEM: You are a Test Reliability Specialist Agent powered by LLaMA. Your responsibility is to analyze test execution history across multiple runs and determine if a test exhibits flakiness.

```

INPUT FORMAT:
- Test Identifier: {test_id}
- Run Outcomes: [{"run_id": int, "result": "PASS"|"FAIL", "timestamp": "ISO
  -8601", "duration_ms": int}]
- Code Base Commit: {git_hash} (verified identical across all runs)

RULES:
1. If all runs PASS -> Deterministic Pass (Not Flaky).
2. If all runs FAIL -> Deterministic Bug / Broken Test (Not Flaky).
3. If runs contain BOTH PASS and FAIL under identical code -> FLAKY.

OUTPUT FORMAT (JSON):
{
  "test_id": "...",
  "is_flaky": true|false,
  "confidence": 0.0 - 1.0,
  "pass_rate": float,
  "flakiness_score": float,
  "failure_modes": ["Timeout", "AssertionError", ...]
}

```

Listing 1: Agent 1: Flakiness Detector System Prompt

```

SYSTEM: You are a Principal Software Engineering Diagnostics Agent. Analyze
the source code and execution logs of an identified flaky test to pinpoint
the precise root cause.

CATEGORIES: [Async_Wait, Concurrency_SharedState, Test_Pollution,
Time_Dependency, Network_External, Unordered_Collection]

FEW-SHOT EXAMPLE:
[SOURCE CODE]:
@Test
public void testAsyncMessageArrival() throws Exception {
    messageProducer.send("payload");
    Thread.sleep(2000); // Wait for processing
    assertEquals(1, consumerQueue.size());
}
[DIAGNOSIS]:
Category: Async_Wait
Root Cause: Test uses an arbitrary fixed sleep (2000ms). Under high CI CPU
load, message ingestion takes >2000ms, causing intermittent assertion
failures.

TASK: Diagnose the following flaky test:
[SOURCE CODE]: {test_source_code}
[FAILURE STACK TRACE]: {stack_trace}

```

Listing 2: Agent 2: Root-Cause Diagnosis System Prompt & Few-Shot Template

```

SYSTEM: You are an Automated Code Refactoring Agent. Given a flaky test and
its diagnosed root cause, rewrite the test to eliminate non-determinism.
Apply standard SE best practices:
- Replace Thread.sleep with condition polling (e.g., Awaitility, await().until
  (...)).
- Inject Testcontainers or wiremock stubs for network calls.
- Clear static state using @BeforeEach / @AfterEach fixtures.
- Mock dynamic clock calls using fixed Instant / Clock injection.

OUTPUT: Provide an explanation of the mitigation strategy followed by a
unified git diff.

```

Listing 3: Agent 3: Code Refactoring Agent System Prompt

Step 4: Concrete Refactoring Demonstration (Unified Diff)

```

--- a/src/test/java/com/service/NotificationServiceTest.java
+++ b/src/test/java/com/service/NotificationServiceTest.java
@@ -14,8 +14,9 @@
     @Test
-    public void testNotificationDelivery() throws InterruptedException {
+    public void testNotificationDelivery() {
+        notificationService.triggerPush("User_123");
-        Thread.sleep(1500); // Flaky: causes race condition under CI load
-        assertTrue(notificationRepository.hasReceived("User_123"));
+        // Mitigated: Poll condition with Awaitility up to 5 seconds
+        org.awaitility.Awaitility.await()
+            .atMost(5, java.util.concurrent.TimeUnit.SECONDS)
+            .pollInterval(100, java.util.concurrent.TimeUnit.MILLISECONDS)
+            .until(() -> notificationRepository.hasReceived("User_123"));
     }

```

Step 5: Synthesized Human-Readable Flakiness Report Sample (Agent 4 Output)

Flakiness Remediation Report: com.service.NotificationServiceTest

Flakiness Verdict: **FLAKY** (Pass Rate: 72.0% across 50 CI executions)

Primary Root Cause: *Async / Wait Timing Dependency*

Identified Code Smell: Hardcoded `Thread.sleep(1500)` on line 16. The background executor occasionally exceeded 1500ms when running on resource-constrained CI runner agents, inducing assertion timeouts.

Mitigation Strategy: Replaced static thread blocking with explicit event-driven condition polling using `Awaitility.await().until(...)`. Configured a 5-second maximum timeout with 100ms polling intervals to handle execution jitter gracefully.

Validation Verification: Refactored test executed 100 times in parallel containerized stress tests with 100% deterministic pass rate.

Solution to Question 10: Step-by-Step BLEU-3 Score Calculation

Final Numerical Answer:

$$\text{BLEU-3} = 0.37128 \quad (\text{Exact: } 0.37128070409594)$$

Given Input Sequences:

- **Reference ($R, r = 10$):** <tok11> <tok8> <tok1> <tok2> <tok16> <tok4> <tok19> <tok15> <tok17> <tok14>
- **Generated ($C, c = 9$):** <tok16> <tok4> <tok19> <tok15> <tok8> <tok14> <tok10> <tok18> <tok6>

Step 1: Sequence Lengths & Brevity Penalty (BP)

- Reference sequence length: $r = 10$ tokens.
- Generated sequence length: $c = 9$ tokens.

Because $c \leq r$ ($9 \leq 10$), the Brevity Penalty triggers:

$$\text{BP} = e^{1-\frac{r}{c}} = e^{1-\frac{10}{9}} = e^{-\frac{1}{9}} \approx e^{-0.111111111111} \approx \mathbf{0.894839316814}$$

Step 2: 1-Gram Precision (p_1) The generated sequence contains $c - 1 + 1 = 9$ unigrams:

$$G_1 = [\text{tok16}, \text{tok4}, \text{tok19}, \text{tok15}, \text{tok8}, \text{tok14}, \text{tok10}, \text{tok18}, \text{tok6}]$$

We check occurrence in the reference token multiset R :

Candidate Unigram	In Ref?	Match	Candidate Unigram	In Ref?	Match
<tok16>	Yes	Matched	<tok14>	Yes	Matched
<tok4>	Yes	Matched	<tok10>	No	Unmatched
<tok19>	Yes	Matched	<tok18>	No	Unmatched
<tok15>	Yes	Matched	<tok6>	No	Unmatched
<tok8>	Yes	Matched			

Matches count $m_1 = 6$. Total candidate 1-grams = 9.

$$p_1 = \frac{6}{9} = \frac{2}{3} \approx \mathbf{0.666666666667}$$

Step 3: 2-Gram Precision (p_2) The generated sequence contains $c - 2 + 1 = 9 - 2 + 1 = 8$ bigrams:

1. (tok16, tok4) $\rightarrow$ Present in reference (indices 5–6) $\Rightarrow$ Match (1)
2. (tok4, tok19) $\rightarrow$ Present in reference (indices 6–7) $\Rightarrow$ Match (2)
3. (tok19, tok15) $\rightarrow$ Present in reference (indices 7–8) $\Rightarrow$ Match (3)
4. (tok15, tok8) $\rightarrow$ Reference has (tok15, tok17), NOT (tok15, tok8) $\Rightarrow$ No match
5. (tok8, tok14) $\rightarrow$ Reference has (tok8, tok1), NOT (tok8, tok14) $\Rightarrow$ No match
6. (tok14, tok10) $\rightarrow$ Not present in reference $\Rightarrow$ No match
7. (tok10, tok18) $\rightarrow$ Not present in reference $\Rightarrow$ No match
8. (tok18, tok6) $\rightarrow$ Not present in reference $\Rightarrow$ No match

Matches count $m_2 = 3$. Total candidate 2-grams = 8.

$$p_2 = \frac{3}{8} = \mathbf{0.375000}$$

Step 4: 3-Gram Precision (p_3) The generated sequence contains $c - 3 + 1 = 9 - 3 + 1 = 7$ trigrams:

1. (tok16, tok4, tok19) $\rightarrow$ Present in reference (indices 5–7) $\Rightarrow$ Match (1)
2. (tok4, tok19, tok15) $\rightarrow$ Present in reference (indices 6–8) $\Rightarrow$ Match (2)
3. (tok19, tok15, tok8) $\rightarrow$ Reference has (tok19, tok15, tok17) $\Rightarrow$ No match
4. (tok15, tok8, tok14) $\rightarrow$ Not present in reference $\Rightarrow$ No match
5. (tok8, tok14, tok10) $\rightarrow$ Not present in reference $\Rightarrow$ No match

6. (tok14, tok10, tok18) $\rightarrow$ Not present in reference $\Rightarrow$ **No match**

7. (tok10, tok18, tok6) $\rightarrow$ Not present in reference $\Rightarrow$ **No match**

Matches count $m_3 = 2$. Total candidate 3-grams = 7.

$$p_3 = \frac{2}{7} \approx \mathbf{0.285714285714}$$

Step 5: Geometric Mean of Precisions Under uniform weights $w_1 = w_2 = w_3 = \frac{1}{3}$:

$$\exp\left(\sum_{i=1}^3 \frac{1}{3} \log p_i\right) = (p_1 \cdot p_2 \cdot p_3)^{\frac{1}{3}} = \left(\frac{2}{3} \cdot \frac{3}{8} \cdot \frac{2}{7}\right)^{\frac{1}{3}}$$

Simplifying the interior product algebraically:

$$\frac{2}{3} \cdot \frac{3}{8} \cdot \frac{2}{7} = \frac{2 \cdot 3 \cdot 2}{3 \cdot 8 \cdot 7} = \frac{12}{168} = \frac{1}{14}$$

Computing the cube root:

$$\left(\frac{1}{14}\right)^{\frac{1}{3}} = 14^{-\frac{1}{3}} \approx \mathbf{0.414913266683}$$

Step 6: Final BLEU-3 Score Synthesis Multiplying the Brevity Penalty by the geometric mean:

$$\begin{aligned} \text{BLEU-3} &= \text{BP} \cdot \left(\frac{1}{14}\right)^{\frac{1}{3}} \\ &= e^{-\frac{1}{9}} \cdot 14^{-\frac{1}{3}} \\ &= 0.894839316814 \times 0.414913266683 \\ &= \mathbf{0.37128070409594} \end{aligned}$$

Reporting with at least 4 decimal digits as requested: **0.37128** (or **0.3713**).

Solution to Question 11: Parameter-Efficient Fine-Tuning & Prompt Tuning

Correct Choice: c. None of the other options is correct

Theoretical Foundations of Prompt Tuning: In **Prompt Tuning** (Lester, Al-Rfou, and Constant, EMNLP 2021, “*The Power of Scale for Parameter-Efficient Prompt Tuning*”):

1. **Freezing the Pre-Trained Architecture:** The *entire* pre-trained language model parameter set Θ_{LM} is kept **completely frozen** (requires $\nabla_{\theta} = 0$). This encompasses:
 - All Transformer layer weights (self-attention projection matrices W_Q, W_K, W_V, W_O , LayerNorm parameters, and feed-forward MLP layers W_1, W_2).
 - The vocabulary embedding matrix $W_e \in \mathbb{R}^{|V| \times d_{\text{model}}}$.
 - The language modeling output projection head (`lm_head`).
2. **Trainable Virtual Tokens:** The *only* parameters updated via gradient descent are a small sequence of l continuous, task-specific virtual prompt vectors (soft prompt tokens)

prepended to the embedded input sequence:

$$P = [p_1, p_2, \dots, p_l] \in \mathbb{R}^{l \times d_{\text{model}}}$$

When processing an input token sequence $X = [x_1, \dots, x_m]$, the input to the Transformer backbone is:

$$\tilde{E} = [P; W_e(X)] \in \mathbb{R}^{(l+m) \times d_{\text{model}}}$$

During backward propagation, gradients are computed solely with respect to P :

$$P \leftarrow P - \eta \nabla_P \mathcal{L}$$

Because the learned soft tokens do not correspond to any vocabulary index in the discrete vocabulary matrix, no component of the pre-trained model itself is modified.

PEFT Method	Updated Parameters	Frozen Components
Prompt Tuning	Continuous soft prompt embeddings (P)	100% of model layers, embeddings, and heads
Prefix Tuning	Virtual prefix keys/values in all layers	Feed-forward networks, embeddings, output head
LoRA	Low-rank adapter matrices ($B \times A$) in attn/MLP	Base weight matrices W_0 , embeddings, heads
Adapters	Bottleneck MLP sub-layers inserted in blocks	Core transformer self-attention and FFN weights
BitFit	Bias terms only throughout the model	All weight tensors and projection matrices
Full Fine-Tuning	All weights, layers, embeddings, and heads	Nothing (100% updated)

Table 7: Systematic taxonomic comparison of Parameter-Efficient Fine-Tuning techniques.

Detailed Refutation of Incorrect Distractor Choices:

- **a is false:** “The adaptation matrices applied to the layers of the model” describes **LoRA** (Low-Rank Adaptation; Hu et al., 2021) or series/parallel Adapter modules (Houlsby et al., 2019), *not* prompt tuning.
- **b is false:** Updating “Only the first transformer layer” is a layer-freezing heuristic, not prompt tuning. In prompt tuning, even layer 0 is completely frozen.
- **d is false:** “A specific classification head appended to the model” describes linear probing or classification fine-tuning. Prompt tuning keeps the native generative LM head frozen.
- **e is false:** The vocabulary embeddings corresponding to natural language tokens (W_e) are kept completely fixed. The prompt embeddings exist outside the discrete vocabulary.
- **f is false:** The final projection layer (`lm_head`) is frozen.
- **g is false:** Updating “All transformer layers and the input embeddings” defines classical **Full Parameter Fine-Tuning**.
- **c is correct:** Because options **a**, **b**, **d**, **e**, **f**, **g** all describe updates to model components, and prompt tuning updates *none* of the model’s internal components, **c** is the only correct answer.